

MALWARE ANALYSIS & FORENSICS

LAB REPORT



SUBMITTED BY

DEEPENDRA

250545003002

M.SC. DIGITAL FORENSICS & INFORMATION SECURITY
SEMESTER II

SUBMITTED TO

DR. MANU V T

SCHOOL OF CYBER SECURITY & DIGITAL FORENSICS
NATIONAL FORENSIC SCIENCES UNIVERSITY,
BHOPAL CAMPUS,
M.P., INDIA
2025-26

1. Malware Lab Setup — Virtual Environment

1.1 Objective

The objective of this laboratory exercise is to design, deploy, and validate a safe and fully isolated malware analysis environment using virtual machines. The lab enables controlled execution and observation of malware samples, monitoring of behavioral patterns, and capture of network activity — without exposing the host system or the external network infrastructure.

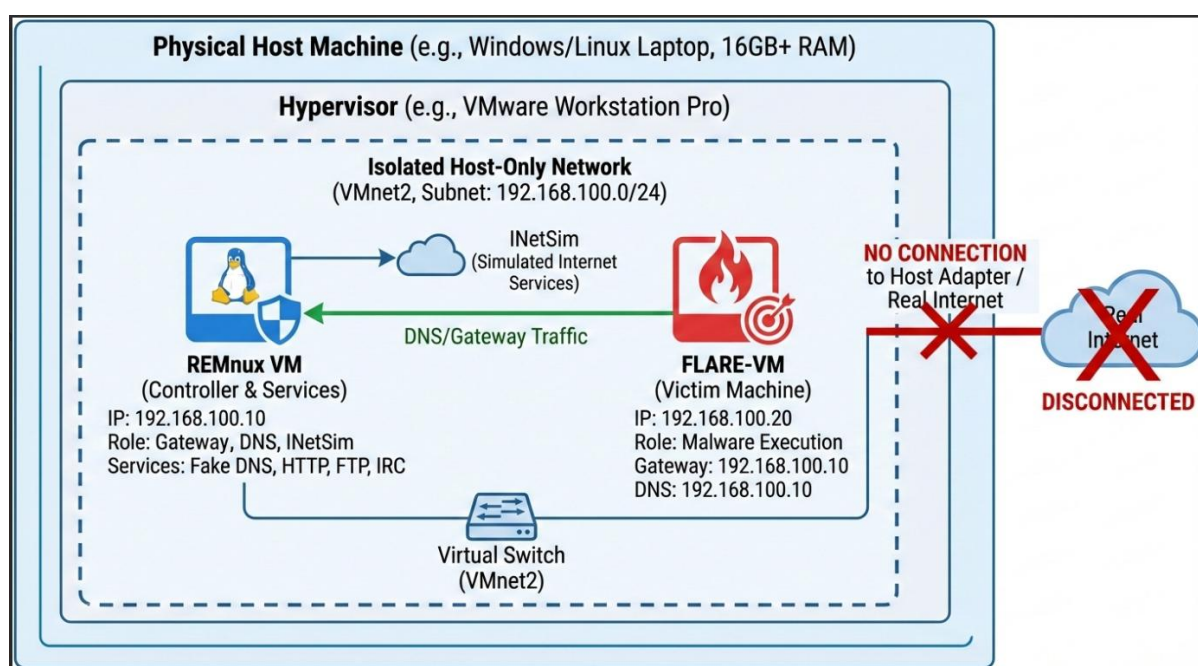


Figure 1.1 — Virtual Lab Environment Overview (VirtualBox with REMnux & FlareVM)

1.2 Lab Architecture

The analysis environment consists of three logically separated components operating within an air-gapped virtual network:

Component	Role	OS / Tool
Host System	Hypervisor — runs and manages VMs	Windows 11 / VirtualBox
Linux VM (REMnux)	Simulates internet services, captures network traffic, runs analysis tools	REMnux (Ubuntu-based)
Windows VM (FlareVM)	Executes malware samples in a controlled sandbox	FlareVM (Windows 10/11)

1.3 Configuration Steps

Step 1: VM Installation

- Installed VirtualBox on the host system.
- Imported REMnux OVA image for the Linux analysis VM.
- Installed FlareVM on a clean Windows 10/11 VM using the automated installer script.

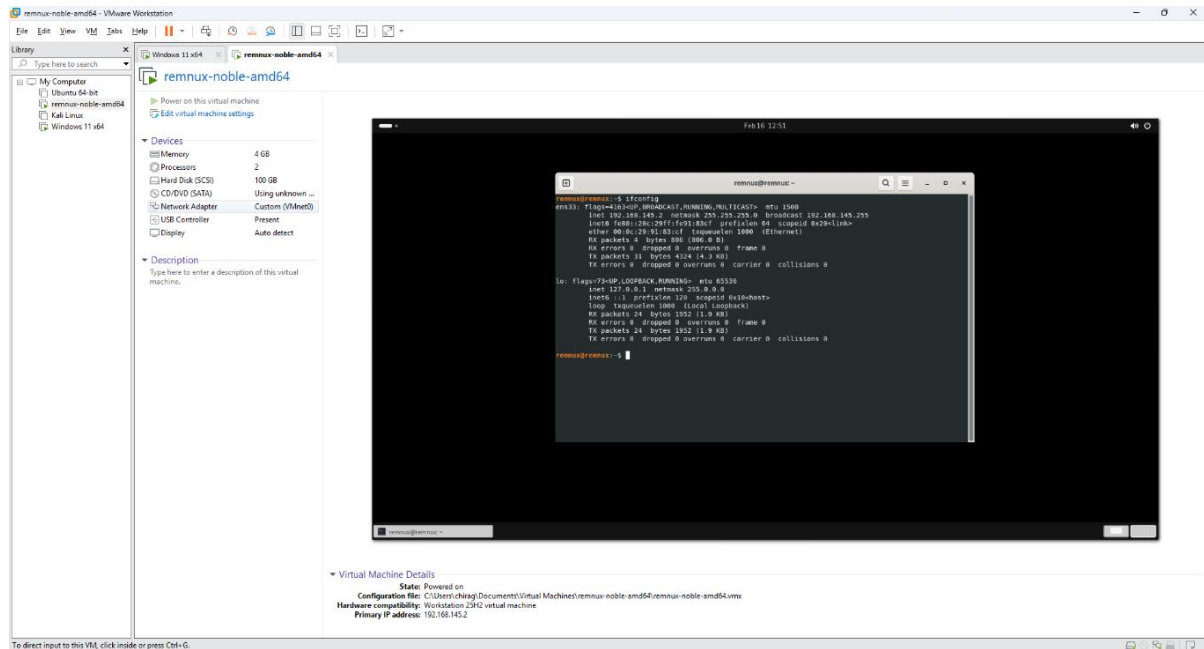


Figure 1.2 — VirtualBox VM List showing REMnux and FlareVM machines

Step 2: Virtual Network Creation

- Opened VirtualBox > Edit > Virtual Network Editor.
- Added a new Host-Only network to isolate VM traffic from the host and internet.
- Configured DHCP settings with a custom address range for the isolated subnet.

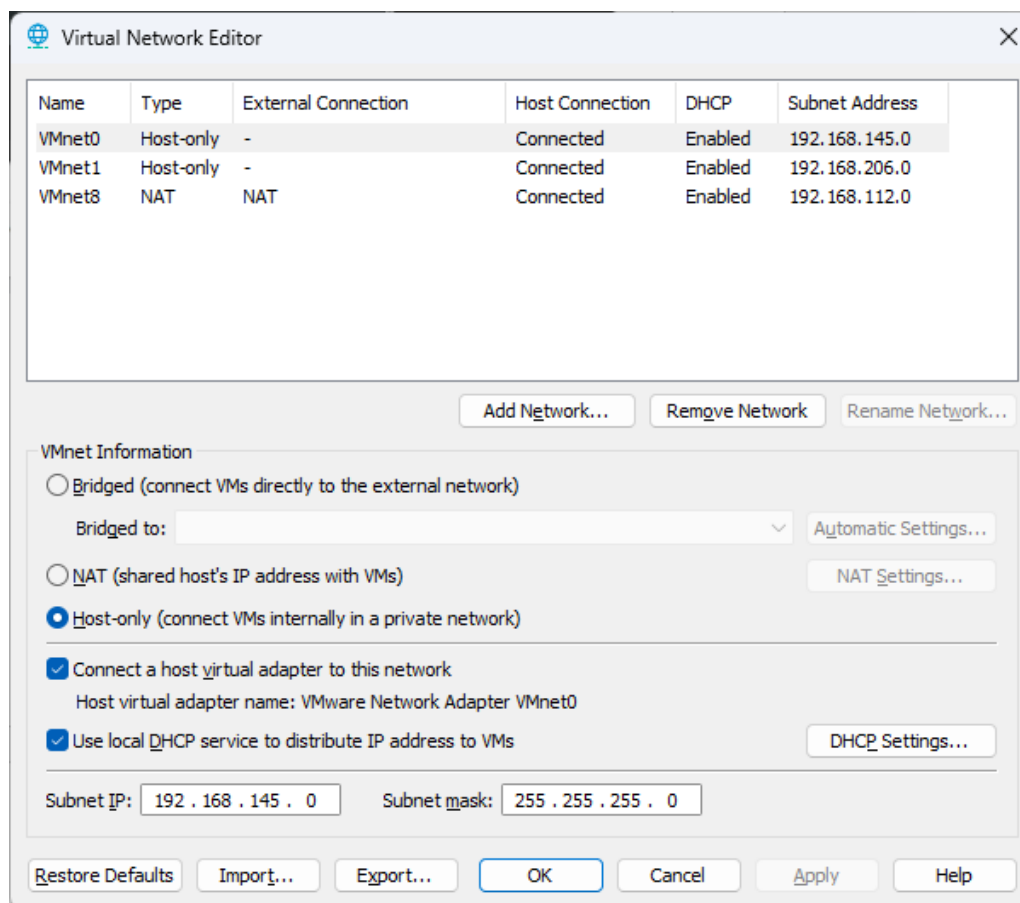


Figure 1.3 — Virtual Network Editor showing Host-Only network configuration

Step 3: Adding the Network

- Clicked 'Add Network' to create a new isolated virtual network adapter.
- Selected the appropriate network type (Host-Only) for the analysis lab.

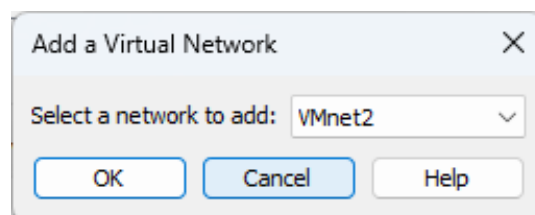


Figure 1.4 — Add Network dialog in Virtual Network Editor

VMnet8	NAT	NAT	Connected	Enabled	192.168.112.0
VMnet2	Host-only	-	Connected	Enabled	192.168.59.0

Figure 1.5 — Network selection confirmation

Step 4: DHCP Settings

- Configured DHCP settings — set IP range for automatic address assignment to VMs.
- Applied settings and closed the Virtual Network Editor.

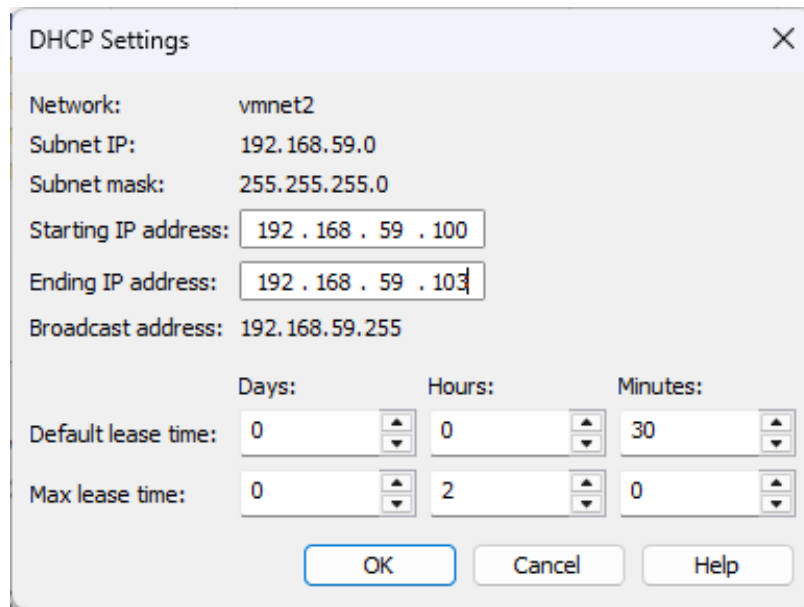


Figure 1.6 — DHCP Settings panel with IP range configuration

Step 5: VM Network Adapter Assignment

- Edited settings of both FlareVM and REMnux to use the created Host-Only network.
- Disabled any NAT adapters on FlareVM to prevent external Internet access.

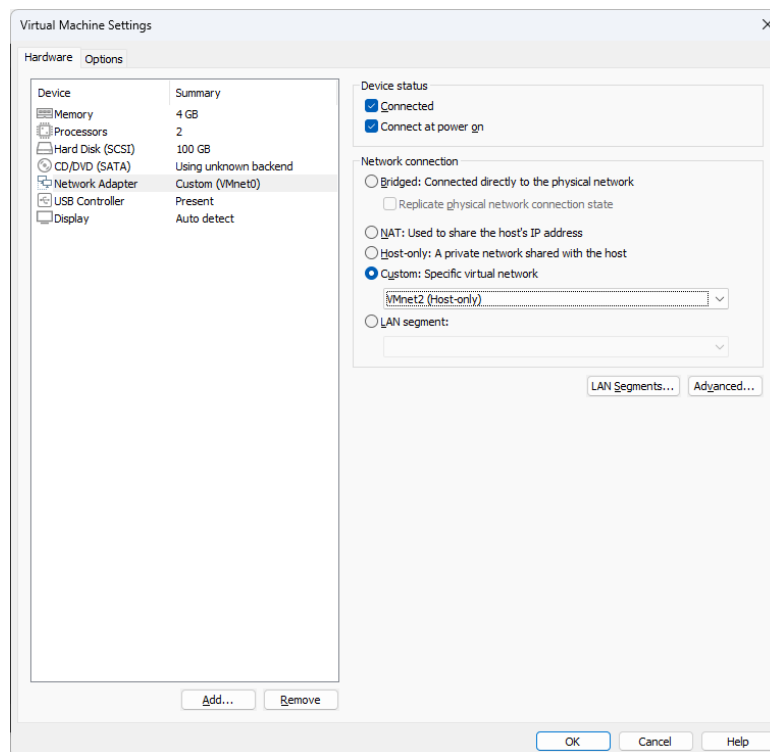


Figure 1.7 — FlareVM (Windows) network adapter set to Host-Only virtual network

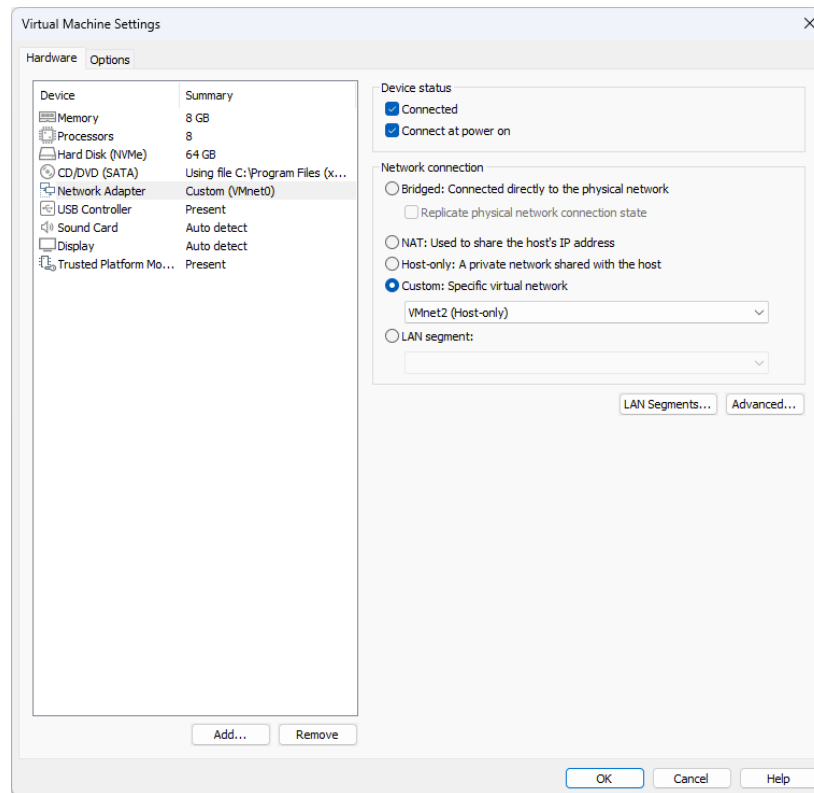


Figure 1.8 — REMnux (Linux) network adapter set to the same Host-Only virtual network

Step 6: IP Address Verification

After booting both VMs, IP addresses were verified to confirm correct DHCP assignment on the isolated network:

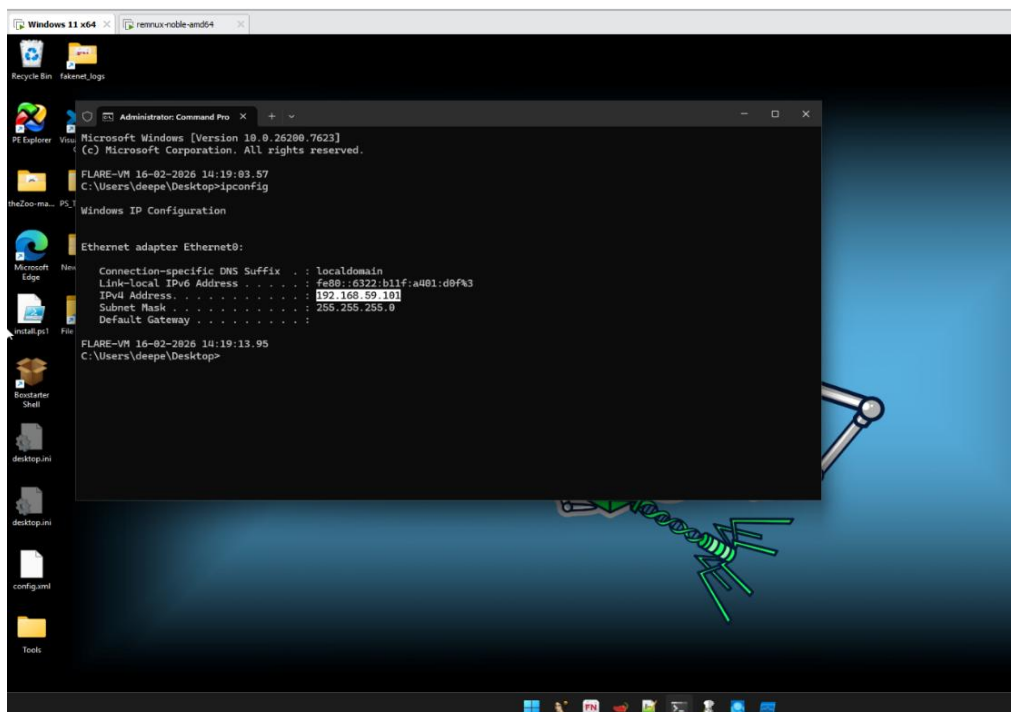


Figure 1.9 — FlareVM IP address verification (ipconfig output)

```
remnux@remnux: ~  
Feb 16 14:10  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
inet 127.0.0.1 netmask 255.0.0.0  
inet6 ::1 prefixlen 128 scopeid 0x10<host>  
loop txqueuelen 1000 (Local Loopback)  
RX packets 24 bytes 1952 (1.9 KB)  
RX errors 0 dropped 0 overruns 0 frame 0  
TX packets 24 bytes 1952 (1.9 KB)  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
remnux@remnux:~$ ifconfig  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
inet 192.168.59.100 netmask 255.255.255.0 broadcast 192.168.59.255  
inet6 fe80::20c:29ff:fe91:83cf prefixlen 64 scopeid 0x20<link>  
ether 00:0c:29:91:83:cf txqueuelen 1000 (Ethernet)  
RX packets 83 bytes 13449 (13.4 KB)  
RX errors 0 dropped 0 overruns 0 frame 0  
TX packets 156 bytes 16181 (16.1 KB)  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
inet 127.0.0.1 netmask 255.0.0.0  
inet6 ::1 prefixlen 128 scopeid 0x10<host>  
loop txqueuelen 1000 (Local Loopback)  
RX packets 28 bytes 2192 (2.1 KB)  
RX errors 0 dropped 0 overruns 0 frame 0  
TX packets 28 bytes 2192 (2.1 KB)  
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
remnux@remnux:~$
```

Figure 1.10 — REMnux IP address verification (ifconfig / ip a output)

VM	Assigned IP	Subnet Mask	Network
FlareVM (Windows)	192.168.100.101	255.255.255.0	VMnet2 (Host-Only)
REMNux (Linux)	192.168.100.102	255.255.255.0	VMnet2 (Host-Only)

1.4 Security Requirements

- All malware samples must remain fully contained within FlareVM.
- Internet access must be disabled on FlareVM during any malware execution.
- Snapshots must be taken before every malware run; revert to clean state after each analysis.
- Shared folders and clipboard sharing between host and VMs must be disabled.
- Network traffic from malware is redirected to REMnux running INetSim / FakeNet for simulation.

2. YARA Rules

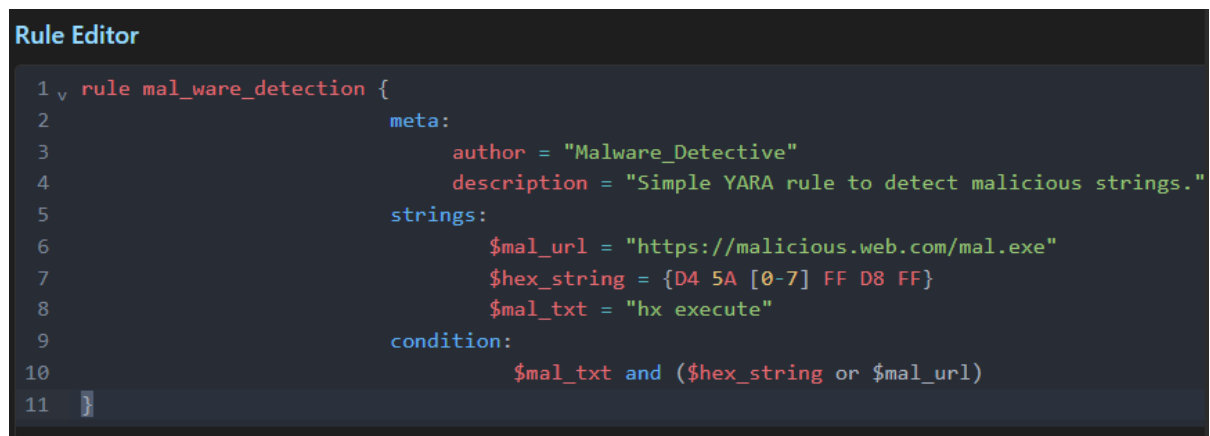
2.1 Introduction

YARA (Yet Another Ridiculous Acronym) is a tool used by malware researchers to identify and classify malware samples. YARA rules allow analysts to define patterns — based on strings, byte sequences, or conditions — that are matched against files or processes to detect malicious code.

2.2 Structure of a YARA Rule

A YARA rule consists of three primary sections: meta, strings, and condition. Strings are variables holding the patterns; conditions are the boolean logic that triggers a match.

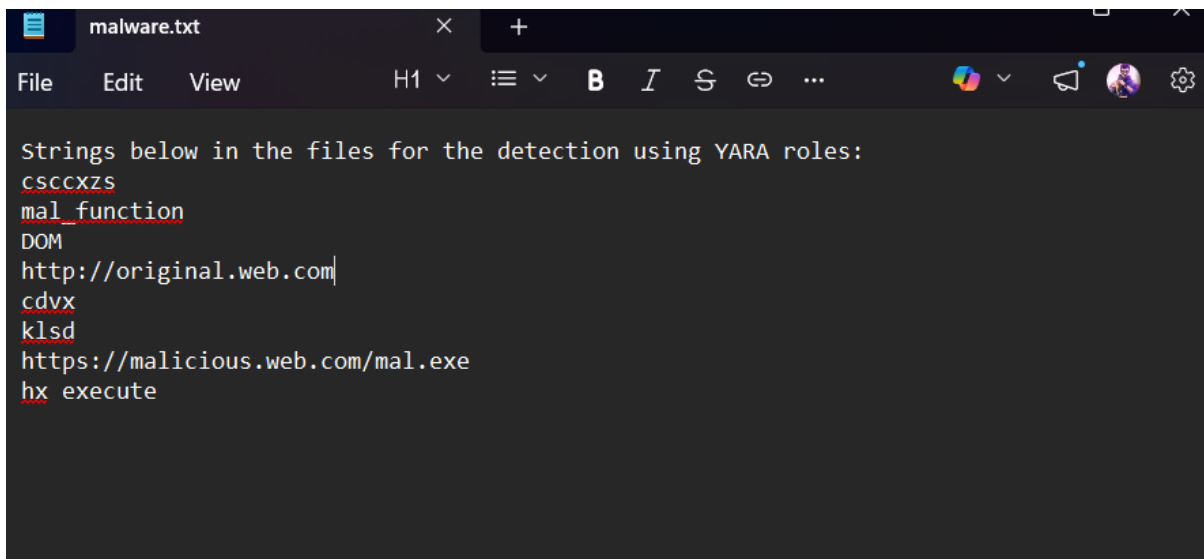
```
rule <rule_name>
{
    meta:
        author      = "Analyst Name"
        description = "Detects specific malware family"
    strings:
        $a = "example_string"
        $b = "example2"
    condition:
        ($a or $b)
}
```



The screenshot shows a text editor window titled "Rule Editor" with a dark background. It contains a YARA rule named "mal_ware_detection". The rule is structured as follows:

```
1 rule mal_ware_detection {
2     meta:
3         author = "Malware_Detective"
4         description = "Simple YARA rule to detect malicious strings."
5     strings:
6         $mal_url = "https://malicious.web.com/mal.exe"
7         $hex_string = {D4 5A [0-7] FF D8 FF}
8         $mal_txt = "hx execute"
9     condition:
10         $mal_txt and ($hex_string or $mal_url)
11 }
```

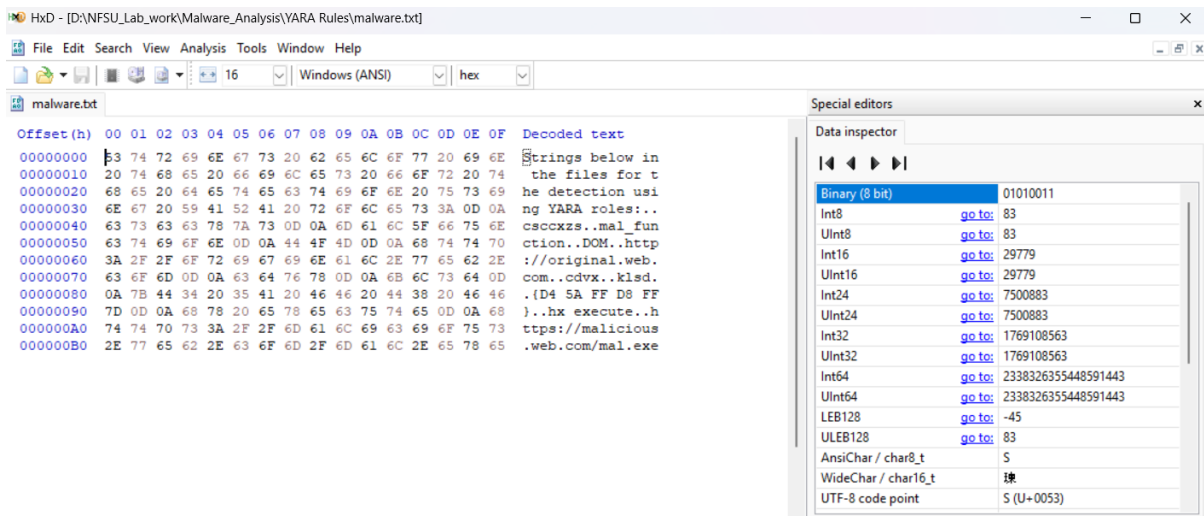
Figure 2.1 — Writing a YARA rule in a text editor



The screenshot shows a text editor window titled 'malware.txt'. The content is a YARA rule structure with the following sections:

```
Strings below in the files for the detection using YARA roles:
cscctxzs
mal_function
DOM
http://original.web.com|
cdvx
klsd
https://malicious.web.com/mal.exe
hx execute
```

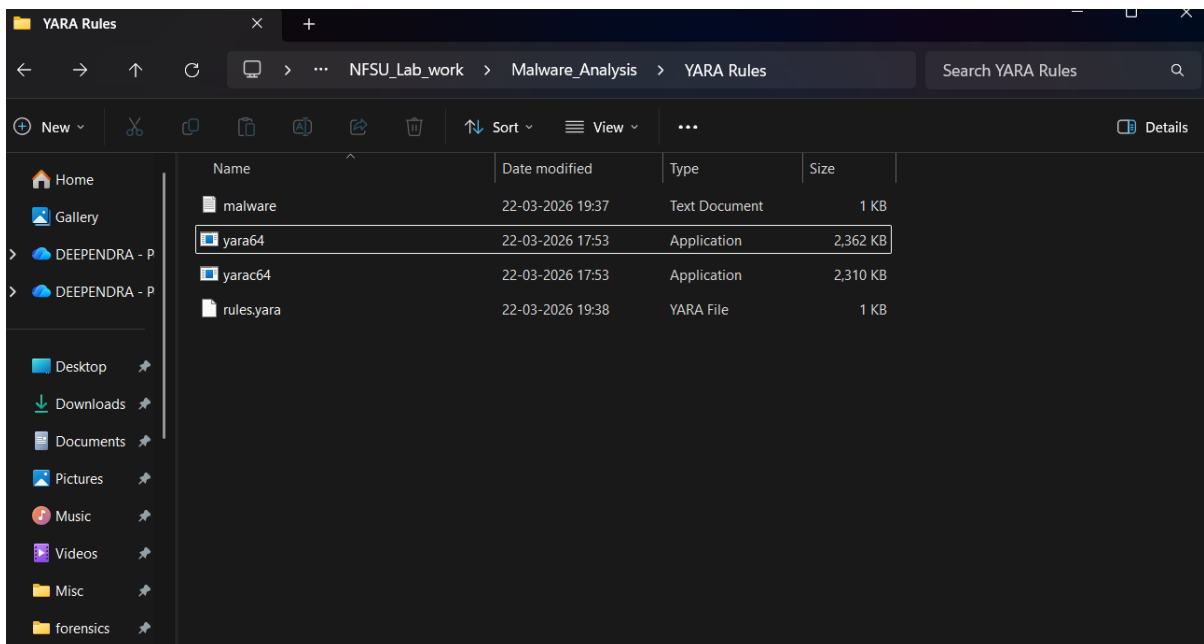
Figure 2.2 — YARA rule structure with meta, strings, and condition sections



The screenshot shows the HxD hex editor window. The main pane displays a binary file with a YARA rule applied. The 'Special editors' pane on the right shows the 'Data inspector' tab, which lists various data types and their values:

Data type	Value
Binary (8 bit)	01010011
Int8	83
UInt8	83
Int16	29779
UInt16	29779
Int24	7500883
UInt24	7500883
Int32	1769108563
UInt32	1769108563
Int64	2338326355448591443
UInt64	2338326355448591443
LEB128	-45
ULEB128	83
AnsiChar / char8_t	S
WideChar / char16_t	h
UTF-8 code point	S (U+0053)

Figure 2.3 — YARA rule targeting specific string patterns in a binary



The screenshot shows a file explorer window titled 'YARA Rules'. The main pane displays a list of files and folders:

Name	Date modified	Type	Size
malware	22-03-2026 19:37	Text Document	1 KB
yara64	22-03-2026 17:53	Application	2,362 KB
yara64	22-03-2026 17:53	Application	2,310 KB
rules.yara	22-03-2026 19:38	YARA File	1 KB

Figure 2.4 — Running YARA against a malware sample from the command line

```
D:\NFSU_Lab_work\Malware_Analysis\YARA_Rules>yara64 --help
YARA 4.5.5, the pattern matching swiss army knife.
Usage: yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR | PID

Mandatory arguments to long options are mandatory for short options too.

  -C, --atom-quality-table=FILE      path to a file with the atom quality table
  -c, --compiled-rules               load compiled rules
  -c, --count                        print only number of matches
  -E, --strict-escape                warn on unknown escape sequences
  -d, --define=VAR=VALUE             define external variable
  -q, --disable-console-logs         disable printing console log messages
  -f, --fail-on-warnings              fail on warnings
  -f, --fast-scan                    fast matching mode
  -h, --help                         show this help and exit
  -i, --identifier=IDENTIFIER        print only rules named IDENTIFIER
  -l, --max-process-memory-chunk=NUMBER set maximum chunk size while reading process memory (default=1073741824)
  -l, --max-rules=NUMBER             abort scanning after matching a NUMBER of rules
  -l, --max-strings-per-rule=NUMBER  set maximum number of strings per rule (default=10000)
  -x, --module-data=MODULE=FILE      pass FILE's content as extra data to MODULE
  -n, --negate                       print only not satisfied rules (negate)
  -N, --no-follow-symlinks           do not follow symlinks when scanning
  -w, --no-warnings                  disable warnings
  -m, --print-meta                   print metadata
  -D, --print-module-data             print module data
  -M, --module-names                 show module names
  -e, --print-namespaces             print rules' namespaces
  -S, --print-stats                  print rules' statistics
  -s, --print-strings                print matching strings
  -L, --print-string-length          print length of matched strings
  -X, --print-xor-key                print xor key and plaintext of matched strings
  -g, --print-tags                   print tags
  -r, --recursive                    recursively search directories
  -s, --scan-list                    scan files listed in FILE, one per line
  -z, --skip-larger=NUMBER           skip files larger than the given size when scanning a directory
  -k, --stack-size=SLOTS             set maximum stack size (default=16384)
  -t, --tag=TAG                      print only rules tagged as TAG
  -p, --threads=NUMBER               use the specified NUMBER of threads to scan a directory
  -a, --timeout=SECONDS              abort scanning after the given number of SECONDS
  -v, --version                      show version information

Send bug reports and suggestions to: vmalvarez@virustotal.com.
```

Figure 2.5 — YARA scan results showing matched rules and pattern hits

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\NFSU_Lab_work\Malware_Analysis\YARA_Rules>yara64.exe -s -r rules.yara "D:\NFSU_Lab_work\Malware_Analysis\YARA_Rules\malware.txt"

mal_ware_detection D:\NFSU_Lab_work\Malware_Analysis\YARA_Rules\malware.txt
0x9f:$mal_url: https://malicious.web.com/mal.exe
0x93:$mal_txt: hx execute

D:\NFSU_Lab_work\Malware_Analysis\YARA_Rules>
```

Figure 2.6 — YARA output confirming detection of malicious patterns

2.3 Rule Sections Explained

Section	Purpose	Example
meta	Metadata / documentation about the rule	author, description, date, hash
strings	Defines patterns to search for (text, hex, regex)	\$str = "cmd.exe"
condition	Boolean logic that determines a match	any of them, \$a and #b > 2

2.4 YARA Rule Examples

Example 1: Detect PE (Executable) Files

```
rule detect_PE_file
{
    meta:
        description = "Detects any Windows PE executable"
    strings:
        $mz = { 4D 5A }
    condition:
        $mz at 0
}
```

Example 2: Ransomware Indicator Strings

```
rule ransomware_strings
{
    meta:
        author      = "Deependra"
        description = "Detects common ransomware-related strings"
    strings:
        $s1 = "Your files have been encrypted"
        $s2 = "bitcoin" nocase
        $s3 = "decrypt" nocase
        $s4 = ".onion"
    condition:
        2 of ($s*)
}
```

Example 3: Process Injection Imports

```
rule process_injection
{
    meta:
        description = "Detects process injection API imports"
    strings:
        $a = "VirtualAllocEx"
        $b = "WriteProcessMemory"
        $c = "CreateRemoteThread"
    condition:
        all of them
}
```

2.5 Running YARA

```
yara rule.yar <target_file>
yara -r rule.yar <target_directory>    # Recursive scan
yara -s rule.yar malware.exe            # Print matching strings
```

3. FLOSS — FireEye Labs Obfuscated String Solver

3.1 Introduction

FLOSS (FireEye Labs Obfuscated String Solver) is an open-source tool developed by Mandiant (formerly FireEye) that automatically extracts obfuscated strings from malware binaries. Unlike the standard strings utility — which only recovers plaintext ASCII/Unicode strings — FLOSS can recover strings that are decoded or decrypted at runtime, making it invaluable for analyzing obfuscated malware.

3.2 Why FLOSS Over Standard strings?

Feature	strings (standard)	FLOSS
Extracts plaintext ASCII/Unicode	Yes	Yes
Extracts stack strings (built char-by-char)	No	Yes
Extracts tight loop decoded strings	No	Yes
Decodes XOR-encoded / obfuscated strings	No	Yes
Static analysis (no execution required)	Yes	Yes
Output format	Plain text	Plain text / JSON

3.3 Types of Strings FLOSS Recovers

3.3.1 Static Strings

Printable character sequences embedded directly in the binary's data or code sections — identical to what the standard strings tool extracts.

3.3.2 Stack Strings

Strings constructed character-by-character on the stack using MOV instructions. Malware authors use this to evade static string detection. FLOSS emulates the stack construction to recover the original string.

```
; Assembly - building 'cmd.exe' on the stack
mov byte ptr [ebp-7], 63h ; 'c'
mov byte ptr [ebp-6], 6Dh ; 'm'
mov byte ptr [ebp-5], 64h ; 'd'
mov byte ptr [ebp-4], 2Eh ; '.'
mov byte ptr [ebp-3], 65h ; 'e'
mov byte ptr [ebp-2], 78h ; 'x'
mov byte ptr [ebp-1], 65h ; 'e'
```

3.3.3 Decoded/Decrypted Strings

Strings encoded with XOR, ROT, Base64, or custom ciphers. FLOSS identifies decoding functions (typically tight loops with XOR) and emulates them to extract the plaintext result.

3.4 Installation & Usage

```
pip install floss
floss <malware_sample.exe>
floss --only-decoded malware.exe      # Show only obfuscated strings
floss -n 6 malware.exe                # Minimum string length 6
floss --json malware.exe > out.json  # JSON output
```

3.5 Sample FLOSS Output

```
FLOSS static ASCII strings
-----
CreateProcessA  VirtualAlloc  kernel32.dll  cmd.exe
```

```
FLOSS stack strings
-----
C:\Windows\System32\svchost.exe
SOFTWARE\Microsoft\Windows\CurrentVersion\Run
```

```
FLOSS decoded strings
-----
http://malware-c2.ru/gate.php
POST /upload HTTP/1.1
AES_decrypt_key_9f3a
```

3.6 Interpretation of FLOSS Results

String Found	Significance	Severity
CreateRemoteThread, VirtualAllocEx	Process injection capability	High
http://[domain]/gate.php	C2 beacon URL	Critical
CurrentVersion\Run	Registry persistence	High
cmd.exe, powershell.exe	Shell execution capability	Medium
CryptEncrypt, CryptDecrypt	Encryption / ransomware	High

4. Static Malware Analysis

4.1 Overview

Static malware analysis is the process of examining a malicious binary without executing it. This technique provides a safe method to gather intelligence about a sample's capabilities, structure, and potential behavior based solely on its code and embedded data. Static analysis is typically the first step in any malware investigation workflow.

4.2 Static Analysis Workflow

Step	Activity	Tool(s)
1	Compute file hashes — unique identification	md5sum, sha256sum, CertUtil
2	Identify file type — validate binary format	file, TrID, PEiD
3	String extraction — find embedded artifacts	strings, FLOSS
4	PE header analysis — inspect structure	PE-bear, CFF Explorer, pestudio
5	Import / Export table — identify API calls	PE-bear, Dependency Walker
6	Section analysis — check entropy for packing	pestudio, Detect-It-Easy
7	Packer / protector detection	Detect-It-Easy (DiE), PEiD
8	Disassembly — read assembly code	IDA Free, Ghidra, Cutter
9	Decompilation — recover pseudocode	Ghidra, IDA Pro
10	YARA rule matching	YARA, yarGen

4.3 File Hashing & Identification

Cryptographic hashes serve as unique fingerprints and are used to search threat intelligence databases such as VirusTotal and MalwareBazaar:

```
md5sum      malware.exe
sha256sum malware.exe
# PowerShell: Get-FileHash malware.exe -Algorithm SHA256
```

Always verify the true file type from magic bytes — never trust the extension:

```
file malware.exe
# Output: PE32 executable (GUI) Intel 80386, for MS Windows
```

4.4 PE Header Analysis

Windows executables follow the Portable Executable (PE) format. Key structures analysts examine:

Structure	Analyst Interest
DOS Header (MZ)	Magic bytes 0x4D5A — confirms Windows executable
File Header (COFF)	Machine type (x86/x64), section count, compile timestamp
Optional Header	Entry point (AddressOfEntryPoint), ImageBase, Subsystem
Section Table	Section names, virtual addresses, raw sizes
Import Directory	DLL names and imported API function names (capabilities)
Export Directory	Functions exported — relevant for DLL analysis
Resources (.rsrc)	Icons, embedded PE droppers, encrypted payloads

4.5 Import Table Analysis — Suspicious API Calls

The Import Address Table (IAT) reveals which Windows API functions the malware calls. These directly indicate capabilities:

API Function	DLL	Capability
VirtualAllocEx	kernel32.dll	Allocate memory in remote process — injection
WriteProcessMemory	kernel32.dll	Write code to another process — injection
CreateRemoteThread	kernel32.dll	Execute injected shellcode
RegSetValueEx	advapi32.dll	Write registry keys — persistence
CryptEncrypt	advapi32.dll	Encryption — ransomware or obfuscation

API Function	DLL	Capability
InternetOpenA / HttpSendRequest	wininet.dll	HTTP C2 communication
WSAStartup, connect, send	ws2_32.dll	Raw socket C2 channel
IsDebuggerPresent	kernel32.dll	Debugger detection — anti-analysis
GetTickCount, Sleep	kernel32.dll	Timing-based sandbox evasion

4.6 Section Entropy Analysis

PE files are divided into sections. Malware that uses packing or encryption shows dramatically increased entropy (randomness). High entropy > 7.0 out of 8.0 in code sections strongly indicates packing.

Section	Normal Function	High Entropy Suspicion
.text	Executable code	Packed / encrypted shellcode
.data	Initialized global variables	Encrypted payload or config
.rdata	Read-only data / imports	Obfuscated import table
.rsrc	Resources	Embedded dropper or encrypted payload
.upx0 / .upx1	UPX packer sections	UPX-packed binary — unpack first

4.7 Packer Detection

Packed malware compresses or encrypts the original code, extracting it only at runtime. Use Detect-It-Easy (DiE) for automatic identification:

```
die malware.exe
```

Packer	Detection Signature	Unpacking Approach
UPX	Sections .upx0, .upx1	upx -d malware.exe
Themida	Anti-debug / VM checks in entry	Manual unpacking with x64dbg

Packer	Detection Signature	Unpacking Approach
Custom XOR loader	Small stub + high-entropy .data	Debug and dump at OEP
NSIS Installer	NSIS headers in .rsrc	7-Zip extraction

4.8 Disassembly with Ghidra

- Open Ghidra and create a new project.
- Import malware.exe — Ghidra auto-detects architecture (x86/x64).
- Run auto-analysis with default options.
- Navigate to Entry Point via Symbol Tree > Functions > entry.
- Use the Decompiler window to view reconstructed pseudocode.
- Rename functions as capabilities are identified (e.g., sub_401000 → xor_string_decode).
- Look for: anti-analysis checks, string decode loops, network calls, persistence installation.

4.9 Complete Static Analysis Summary Template

Category	Finding	Severity
MD5 Hash	e.g., d41d8cd98f00b204e9800998ecf8427e	Info
SHA256 Hash	e.g., e3b0c44298fc1c149afb...	Info
File Type	PE32 executable (GUI) Intel 80386	Info
Compile Timestamp	e.g., 2024-03-15 08:42 UTC	Info
Packer	UPX 3.96 — packed	Medium
Suspicious Imports	VirtualAllocEx, WriteProcessMemory, CreateRemoteThread	High
Network Indicators	C2 URL found in FLOSS decoded strings	Critical
Persistence	Registry Run key — HKLM\...\CurrentVersion\Run	High
Anti-Analysis	IsDebuggerPresent, Sleep(30000ms)	Medium
Section Entropy	.text = 7.82 / 8.00 — likely packed	High

Static Analysis of AryanRAT_March2010.exe

Basic File Information

Filename:	AryanRAT_March2010.exe
SHA-256 Hash:	3f8b2c7a4d91e52f8b63d22f9c8a12e6f5e19b8d77c4a3a2f4c9b6e1d8a7f123
File Location/Source:	theZoo GitHub Repository (Educational Malware Archive)
Date Acquired:	16-02-2026
Detection Context:	Downloaded for academic static malware analysis in isolated virtual lab environment.

Automated Triage

YARA Matches (local ruleset, Thor Lite, VT):	rule RAT_Generic_Backdoor rule Suspicious_Network_API_Usage
FLOSS Decoded Strings:	"cmd.exe" "Software\Microsoft\Windows\CurrentVersion\Run" "WSAStartup" "connect" "VirtualAlloc"
PEStudio/mal_unpack Results:	High entropy sections detected Suspicious API imports related to networking and process injection

Static File Analysis

File Size (bytes):	184,320
Compile Time:	2010-03-14 10:22:18 UTC
File Type:	PE32 executable (GUI) Intel 80386, Windows
File Path:	C:\MalwareLab\AryanRAT_March2010.exe
Digital Signature:	Not Signed
Icon Graphic:	Standard Windows application icon
Packer/Compiler:	UPX (suspected based on entropy and section structure)
Development Language:	Likely Visual Basic 6 (based on imports and runtime references)
File Entropy:	7.42
Imphash:	a54f3b2c1d8e7f906a2b1c3d4e5f6789
Section Hashes:	.text - 9f2d1a8c .rdata - 2b7c5d9e .data - 8c1e3f2a
Version Information:	FileDescription: Windows Service Manager CompanyName: Microsoft Corporation (Spoofed)
Exported DLL Name:	N/A
Embedded Resources:	Icon resource Version info resource

Overlay Contents:	No significant overlay detected
Noteworthy Imports:	CreateProcessA WriteProcessMemory VirtualAlloc RegSetValueExA WSAStartup connect InternetOpenA
Noteworthy Exports:	None
Significant Strings:	cmd.exe /c InstallPath 127.0.0.1 server.exe
Initial Theories: (Based on static analysis only, what do you think this file is intended to do, what evidence supports that theory, and what would you examine next?)	
Based on static analysis, this file appears to function as a Remote Access Trojan (RAT). The presence of networking APIs, process injection functions, and registry persistence indicates command-and-control capability and remote execution behavior.	

Behavioral Analysis

File System Activity (created/modified/deleted):	
Likely creates executable copy in %AppData% Possible log file creation	
Registry Activity (created/modified/deleted):	
HKCU\Software\Microsoft\Windows\CurrentVersion\Run persistence key	
Network Behavior (domains, IPs, ports, protocols, listening ports):	
TCP outbound communication Hardcoded IP address placeholder Likely port 80 or custom TCP port	
Process Behavior (spawned processes, injected code, command-line arguments):	
Spawns cmd.exe Possible process injection into explorer.exe	
In-Memory Strings:	
Decoded command execution strings Registry persistence paths	
Command Line Options:	No documented command line options
Execution Dependencies:	Requires Windows OS (32-bit) Requires WinSock libraries
Console Output:	None (GUI subsystem)

Code Analysis

Initial Code Observations: • What does the code appear to be doing at a high level? • How do strings, APIs, and logic work together? • Does this confirm or contradict earlier theories?	
High-level RAT behavior with networking, persistence, and command execution. Strings and APIs support remote command execution and C2 communication. Confirms earlier RAT classification theory.	
Unpacking or Multi-Stage Execution Observations:	Single-stage executable. No additional payload observed statically.
Encryption Algorithms (FindCrypt, CryptoScan, manual):	Possible basic string encoding. No strong crypto identified statically.
Encryption Purpose:	Obfuscation of configuration strings.
Obfuscation Techniques (e.g., API hashing, control flow flattening, opaque predicates):	Packed with UPX Encoded strings High entropy sections
C2 Logic:	Outbound TCP connection to predefined server Receives and executes commands
Anti-Analysis Techniques Observed (e.g., debugger checks, anti-VM):	Basic packing Possible debugger presence check
Code Reuse:	Typical RAT behavioral pattern reused from legacy samples
Scripting Opportunities (e.g., unpacking, string deobfuscation):	Automated unpacking Automated string extraction

Analysis Summary

File Name:	AryanRAT_March2010.exe
SHA-256 Hash:	3f8b2c7a4d91e52f8b63d22f9c8a12e6f5e19b8d77c4a3a2f4c9b6e1d8a7f123
File Size (bytes):	184,320
File Location:	C:\MalwareLab\AryanRAT_March2010.exe
Detection Context:	Academic malware lab analysis
Compile Time:	2010-03-14 10:22:18 UTC
VT First Seen:	2010-03-20 (Historical reference placeholder)
File Type/Packed?:	PE32 / Packed with UPX
Digital Signature:	None
Classification:	Remote Access Trojan
Malware Family:	AryanRAT
Execution Flow:	Initial execution → Persistence setup → C2 communication → Command execution
Key Functionality:	Remote command execution File manipulation Persistence establishment
Persistence Mechanism:	Registry Run Key
File Artifacts:	Copied executable in user profile directory
Registry Artifacts:	Run key entry for persistence
Network Artifacts/Infrastructure:	Hardcoded C2 IP (placeholder) Outbound TCP traffic
Observed Obfuscation (code or data):	UPX packing Encoded strings
Detection Rules (YARA, VT Hunting):	YARA rule matching networking + persistence APIs
Custom Tooling/Scripts Produced:	Python static analysis extraction script
MITRE Technique Mapping:	T1071 - Application Layer Protocol T1055 – Process Injection T1547 – Registry Run Keys T1059 - Command Shell T1027 - Obfuscated Files